OPEN BOOK · COMPLETE REPLICATION & METHODOLOGY REFERENCE

Open Book

How the desk tool turns a balance sheet into goals, ideas into client-specific recommendations, and research into a scored daily feed — every formula, every screen, and the whole daily process, end to end. Enough to replicate the project from scratch.

Prepared
12 August 2026

Audience
The detail-oriented reader — and anyone replicating the project from scratch

Source of truth
data.js · goals.js · mapping.js · scanner.js · expressions.js · email.js · morgan.js · charts.js · build_today_focus.py · tv_technicals.py · fetch_chart_series.py · openbook.js · app.js

Nature
Static client-side app · no build step · all logic is pure functions · live at open-book-eta.vercel.app

> **The one-sentence version.** Open Book is a private-bank desk tool that **infers** each client's goals from their balance sheet, **derives** (never hand-picks) which investment ideas fit which clients through a single transparent scoring engine, and **scores** each idea's standalone conviction from a research feed — so every recommendation on screen can be opened up and traced back to a number and a reason.

This document is exhaustive by design. Section 1 sets out the goal of the project and its mental model; section 2 walks the live app screen by screen. Section 3 covers the data the whole thing runs on. Sections 4–8 are the engines, with every calculation written out. Sections 9–11 cover where ideas come from — including the full daily-sweep discipline — and the supporting analytics. Section 12 runs one idea through the whole machine. Sections 13–16 cover calibration, the daily operating loop, design & deployment, and the replication checklist. Every formula here is copied from the live code, not paraphrased.

The whole machine on one page

Before the detail, here is the entire system in plain English — every section later in this document expands one of these rows.

Step	What happens	In plain terms
1 · Research	A daily market sweep writes a data file of trade ideas, each with sourced facts, a one-sentence recommendation, and honest tags on anything estimated (§9.2, §14)	"Here is what the desk thinks today, and here is the evidence."
2 · Score the idea	A generator grades every idea against a published rubric and computes a conviction score out of 100 (§8)	"How good is this idea, on its own merits?"
3 · Know the client	An inference engine reads each client's balance sheet — goals, liabilities, behaviour — and derives what they actually need: Growth / Income / Preservation percentages (§4)	"What does this client really want, whatever the form says?"
4 · Match them	A single fit engine scores every idea against every client on four transparent axes, gated by what the client is legally allowed to trade (§5–7)	"Is this idea right for *this* person — and can they even trade it?"

5 · Curate	The Solutions desk approves, discards or rewrites what the sweep produced; only approved ideas reach advisors (§2.4)	"A human signs off before anyone sees it."
6 · Act	The advisor sees a ranked feed, opens the per-client suitability read, and drafts a personalised email — all from the same live numbers (§2.2–2.3, §11)	"From idea to client conversation, with the reasoning attached."

> **The two numbers to keep apart.** Every idea carries a **conviction** score ("how good is the idea?") and, per client, a **fit** score ("how right is it for them?"). They are computed by different engines, shown side by side, and never blended — a brilliant idea can be wrong for a cautious book, and a modest idea can be exactly what an under-allocated client needs.

Speaking the desk's language — a short glossary

Term	Meaning here
Bucket	One of three goal roles money can play: Growth (make it), Income (pay me), Preservation (don't lose it). Both clients' goals and ideas are expressed in these.
Mandate	The client's overall stance — growth / income / preservation — parsed from their stated risk profile.
Expression / structure	*How* a view is implemented: directly (buy the stock), as a packaged note (autocall, buffered note), or as an OTC derivative (collar, call spread, FX forward).
Structured note vs OTC	The key legal split: packaged *notes* are securities a Retail client can buy; bilateral *OTC derivatives* require Professional classification. One keyword classifier encodes this (§3.6).
Pre-position / post-print	An earnings idea's stance: entering *before* a company reports vs trading the *reaction* after a verified result.
Tactical	A short-dated option trade with explicit entry / target / stop levels and a stated trigger — it only appears on the board while its trigger is actually met (§9.2).
Sourced / estimated	Every datapoint's honesty tag: confirmed by two independent sources, or not. Estimated inputs cap how high an idea's conviction label can go (§8.3).
Own view	An idea the desk built without any outside research to cite — labelled as such rather than dressed with borrowed authority.
Posted / updated	Card dates driven by a fingerprint ledger: "posted" only moves when an idea's *argument* changes; "updated" marks a re-grounding (§9.2).
Flagged / suppressed	A client the fit engine recommends an idea to — versus one blocked by the MiFID gate, always shown *with the reason* rather than hidden.

Contents

1 · The goal of the project

The Brokerage Playground is a **client-focused desk tool** for a private-bank investment advisor, restyled in a **J.P. Morgan Private Bank** aesthetic (espresso and bronze, serif display type, hairline rules). It is a static website — no server, no build step, no database. Everything is a **pure function over seed data** held in the browser, which is the whole point: every recommendation is **reproducible and inspectable**, never a black box and never hand-curated.

It exists to answer four questions an advisor faces every morning, and to answer them *transparently*:

1. **What does this client actually want?** — not what a form says, but what their balance sheet and behaviour imply. (Goal inference.)
2. **Of the desk's standing ideas, which fit *this* client, and how well?** — with a per-axis breakdown you can open up. (The fit engine.)
3. **Of today's market events, which are worth acting on, and for whom?** — a daily sweep scored for conviction and mapped to the live book. (Today's Focus.)
4. **If I put this idea in front of *this* client, is it appropriate — and how do I say it?** — the suitability read (MiFID gating with reasons shown) and the book-aware email. (Suitability & email.)

> **The governing principle:** *idea→client links are derived, never hand-picked, so the views can't drift.* If you change a client's book, the clients an idea is "flagged" to recompute automatically — there is no hand-maintained mapping table that can fall out of sync with reality.

Two distinct axes the tool keeps separate

A recurring source of confusion in advice tools is conflating "is this a good idea?" with "is this idea right for this client?" The Playground keeps them on **two orthogonal axes**:

Axis	Question	Range	Computed by
Conviction	How good is the idea, on its own merits?	0–100 + tier	`build_today_focus.py` (rubric)
Client-fit	How right is this idea for *this* client's book and goals?	0–100 + tier	`mapping.js` (4 axes × gate)

A high-conviction idea can be a poor fit for a conservative book; a modest idea can be exactly what an under-allocated client needs. The two scores are shown side by side and never blended into one misleading number.

2 · The app — what the user sees INDEX.HTML · OPENBOOK.JS

The live app is a **redesigned shell** over the original engines: one page, two header **views** (Advisor · Solutions) crossed with two **tabs** (Idea Feed · Advisor Book). The original four-tab app survives intact as `classic.html` and is embedded for the deep client pages (§2.5), so nothing was thrown away — the shell is a new front door onto the same machinery.

2.1 The shell

A sticky header carries the **"Open Book · PRIVATE BANK"** lockup and, on the right: the **Advisor / Solutions view switch** (who you are — §2.4), a **light / dark theme toggle** (persisted in `localStorage` under `ob-theme`, applied by an inline script *before first paint* so there is no flash, and synced into the embedded iframe and across open pages via storage events), a **"Search everything ⌘K"** command palette across ideas and clients, and the **as-of stamp** read from the sweep file. Below it, the two tabs.

2.2 The Idea Feed

The daily sweep rendered as a **social-style origination feed**. At the top sits the **Market Brief** card — the sweep's summary paragraph, expandable to its "to watch today" bullets and full narrative. Below it, a three-column layout:

Column	What lives there
Left rail	**TOP 3** — three tiles ranked for *your* book: by **breadth** first (how many of your clients the live fit engine flags the idea to), then desk conviction; shown in conviction order. A sweep idea flagged `pinned` leads the board as a **DESK PICK** — a label only; its scores are untouched. Tapping a tile focuses the feed on that idea; tapping again clears. Underneath, the **CLIENT TOOLKIT** — tap a client to open their action list and build their email.
Middle	The feed itself: a search box ("gold", "MU", "autocall" or a client name), asset-class filter chips, a client dropdown (filters to ideas suited to that client), a count line, and the idea cards (§2.3).
Right rail	**Ask your book** — the Morgan AI panel. It answers questions about clients, ideas and risk *from the live engines only* (per-client fit, conviction, comfort limits, goal derivation) — "it never invents a number" (§11.2 / `morgan.js` + `methodology.js`).

2.3 Anatomy of an idea card

Every idea renders as a post, and every element on it is derived, not typed:

- **Author identity** — from provenance: an on-theme sweep idea (`themeId` set) posts as *Solutions Desk*; an off-theme one as *Claude's Views*; an advisor draft as *You — Solutions view*.
- **"Posted Nd ago"** — reads the idea's *own* `postedAt`, stamped by the honest-date ledger (§9.2), never the global sweep date. A separate **"updated Nd ago"** chip appears only when the idea's argument was re-grounded after first posting.
- **The CONV ring** — the idea's conviction score /100 from the rubric of §8, drawn as an SVG arc. Clicking it opens the full scoring breakdown (pillars, data-quality tags, the banding). This is the *idea* axis; per-client fit deliberately lives elsewhere (§1).
- **Ticker + asset tags, name, thesis** — the argument itself.

- **The RECOMMENDATION block** — the idea's one-sentence `tradeStatement` (instrument + direction + the view). If the Solutions view has tweaked the implementation (§2.4), the tweaked text shows here — and feeds the email engine — with a "tweaked by Solutions" marker.
- **An inline chart** — only when the sweep shipped a *sourced* series (§9.2); no data, no graph.
- **The "Via ..." line** — lists only the named authors whose work was *actually read* (sources of kind `view`). An idea that is entirely the desk's own call shows *"Own view — no outside research leaned on"* instead. Borrowed authority is structurally impossible: the generator rejects a view-citation without a reading basis (§9.2).
- **♥ Save** + engagement counters (liked / drafted / checked by N advisors — presentation-layer counters persisted in `localStorage`).
- **Two actions: Idea Suitability** — the per-client flowchart: every flagged client with their fit score, tier and a plain-English "why this was flagged for X" (holdings hook, mandate fit, goal-gap line), plus the suppressed clients *with their MiFID reason*; and **Email** — the book-aware letter composer (§11.3).

2.4 The Solutions view — curating the feed

The view switch is a **role switch**. The Advisor view sees only **approved** ideas and zero curation UI. The Solutions view sees the whole board and manufactures it:

Control	What it does
✓ **Approve** / ✕ **Discard**	The gate on every sweep card. Only approved ideas reach the Advisor view (approved is the default; a discard keeps the idea visible in Solutions, greyed, with "Hidden from the Advisor view"). Custom drafts are locked approved.
✎ **Edit implementation**	Rewrites the recommendation text the advisor cards and emails use, per idea — the desk's sign-off layer over the sweep's default wording.
+ Add idea	The composer: name, ticker, asset class, goal bucket, direction, thesis, a one-sentence recommendation, comma-separated expressions. The draft joins the feed under the advisor's own name, is **not desk-scored** (a DRAFT pill instead of a conviction ring — the rubric only scores the sweep), but maps to clients through *exactly the same live fit engine* as every other idea.
✕ **Remove** / ↺ **Restore**	Takes a desk idea off the feed (or deletes a draft); removed ideas are restorable in one click.

All curation state persists in `localStorage` (`ob_ui_v1`: approvals, implementation edits, engagement counters, the chosen view; a user store holds custom drafts and removals) — so the demo behaves like a stateful product with no server.

2.5 The Advisor Book — and the embedded legacy pages

The second tab is **coverage at a glance**: a grid of client tiles. Tapping a client hides the grid and embeds that client's **current-portfolio report** — `portfolio.html?embed=1&client=…` — inside the shell with an "← All clients" bar: the allocation-vs-goal bar, the desk agenda, the ideas mapped to them with reasoning, holdings, and the "how were these goals derived" panel that exposes the full §4 arithmetic. The legacy pages detect `embed=1`, hide their own masthead and tab bar, add a back pill, and inherit the shell's theme.

The **original app** lives on as `classic.html` (+ `app.js`) — kept so the embedded client pages and the draft-a-view flow keep working; it is not part of the live navigation. `onepager.html` renders a

printable client one-pager. Everything in §§3–11 powers both front ends identically, which is the point: the engines don't know or care which UI is asking.

3 · The data model DATA.JS

Three collections seed everything: **themes**, **ideas**, and **clients**. They are exposed on `window.SEED`. Understanding their fields is a prerequisite for every formula that follows.

3.1 Themes

Nine standing themes (`SEED_THEMES`), each `{ id, name, blurb }` : AI & Productivity, Power & Infrastructure, Core Fixed Income, Broadening Equity, Real Assets, Resilience & Protection, Gold & Currency, Healthcare Innovation, Structured Outcomes. Themes are the "house view" buckets; an idea's theme membership later feeds the conviction house-view pillar.

3.2 Ideas

Each idea (`SEED_IDEAS`) is tagged on four independent dimensions plus copy:

Field	Values	Role
`type`	Thematic · Opportunistic · Strategic	The idea's character / horizon style
`assetClass`	Equity, Fixed Income, Real Assets, Commodity, Alternatives, Structured, Multi-Asset	What instrument class it lives in
`sector`	Technology, Utilities, Energy, Gold, Rates, Credit, Broad, …	Sector signal for the fit axes
`bucket`	Growth Income Preservation	Which of the three goal buckets the idea *serves*
`conviction`	High · Medium-High · Medium	The desk's standing conviction (Views ideas)
`structures`	e.g. ["Direct equity", "Zero-cost collar", "Buffered note"]	How to express it; `structures[0]` is the *natural expression*
`intent`	add · protect · trim · income	Legacy/display field; no longer drives the engine
`tickers`	optional alias list, e.g. gold = [XAU, GLD, 4GLD, …]	Lets the engine recognise the idea in a book by name

The seed carries roughly 16 ideas across the nine themes — from "AI infrastructure & compute leaders" (Equity / Technology / Growth) to "Gold as a debasement hedge" (Commodity / Gold / Preservation) to the desk's flagship structured note, a HALO equal-weight basket autocall paying a fixed 24% p.a. coupon with a 20% cushion.

3.3 Clients — the nine books

Nine deliberately different books (`SEED_CLIENTS`), each engineered so the scanner and the fit engine surface a *different* set of actions. Each carries: `aum` , `ccy` (base currency), `classification` (Retail / Professional) and `mifid` , a free-text `risk` string, a `split` (asset-class weights), an array of

`positions`, optional `liabilities`, and a `goals` object holding a free-text objective, horizon, and (sometimes) a `funding` headline.

Client	AUM	Class	Stated risk	The teaching point
Fable	$38m	Professional	Aggressive	Concentrated AI book, options-fluent
Aurora (real anchor)	€50.2m	Retail	Growth + income	25.8% Micron on a ~4× gain; 72% USD vs EUR base
Scott	$124m	Retail	Conservative income	Large diversified retirement/preservation estate
Amar	$71m	Professional	Growth, real-asset tilt	Real assets + a 16% Bitcoin position down ~35%
Jacob	$96m	Professional	Moderate	Balanced book + a $19m mortgage & tax bill (liabilities)
Prahnav	$44m	Retail	Growth	Buckets on-target, but two concentrated winners; Retail collar problem
Ben	$29m	Retail	Moderate, value tilt	82% equity, thin income/protection
Mitch (was Morgan)	$38m	Professional	Aggressive — max growth	**No funding goal & no targets** — goal must be inferred from the book
Fotis (was Tejpaul)	$60m	Retail	Not profiled	**No mandate, no goal, never risk-profiled** — read purely off the book

Mitch and Fotis are the deliberate "nothing on file" cases that prove the goal-inference engine earns its keep. Aurora is the real anchor book; the others are consistent, distinct private-bank books.

3.4 The reconciliation invariant

> `positions` is the single source of truth. Each book's `weightPct` values **sum to 100**, and each client's `split` literal **equals the per-asset-class sum of its positions** (`split ≡ Σ positions`). So whether a calculation reads the split or folds the positions, it sees the same book.

3.5 Synthetic 24-month sector history

The books only carry *current* holdings, but the Affinity axis (§6.2) needs history. So `data.js` synthesises, per client, a **24-month monthly sector-allocation series** (`client.sectorHistory[sector]` , index 0 = most recent = current). It is **deterministic** (no RNG) and **clearly labelled synthetic seed data**. The trajectory is shaped from each sector's dominant-position P&L as a proxy for how the weight got there:

Shape	Trigger (top position P&L)	Trajectory toward "now"	Example
ramp	winner, pnl ≥ +50%	starts low (~12% of current), rises into the book	Micron in Aurora
fade	loser, pnl ≤ −10%	starts high (~130% of current), drifts down	an underwater bond

core	else	roughly steady around current (~92% start)	an index core

Invariant: `history[0]` exactly equals the live current allocation, so the synthetic past is always consistent with the real present. To go live, replace the array with a true monthly series — nothing else changes.

3.6 MiFID complexity classification

The single most consequential domain rule. Every structure string is classified by `complexityOf()` into one of three buckets via keyword match — **order matters** (structured is checked before OTC, so "HALO basket (ACM+)" reads as a note, not a plain basket):

Class	Examples	Retail can trade?
structured	autocall, Phoenix, reverse convertible, buffered note, capital-protected note, certificate, HALO	**Yes** — a packaged *security* (still complex → appropriateness applies)
otc	collar, risk reversal, accumulator, prepaid variable forward, FX forward, covered call, protective put	**No** — a bilateral OTC *derivative*; needs Professional re-classification
non-complex	direct equity, ETF, index core, bond ladder, fund	**Yes**

This single classifier drives the global **tradability gate** (§7.1) and every appropriateness read in the app. The distinction it encodes — packaged structured *notes* are Retail-eligible, OTC *derivatives* are not — is the heart of the app's suitability logic.

4 · Goal inference — `deriveGoals()` GOALS.JS

Real private-bank clients never hand you a clean target allocation. They say "I want to grow capital," "I have a $19m mortgage," "I need $0.8m/yr." The Playground therefore **infers** a goal from *both sides of the balance sheet* plus revealed risk appetite, into three buckets:

PRESERVATION INCOME GROWTH

The methodology is liability-driven / goals-based (in the spirit of Chhabra's risk-allocation framework): **fund the floor first** (obligations are not preferences), then **deploy the surplus** by how hard the money must work. Every coefficient is a calibration choice and lives in `PARAMS`; every *input* is a real measured quantity off the balance sheet.

> **Anti-circularity (the crucial design choice).** The current book sets only a *willingness knob* (a single scalar), **not** the target vector. The target is still built bottom-up by the bucket logic, so the gap between what the client *has* and what they *need* stays meaningful. Without this, the model would always report "you're on target."

4.1 Step 1 — measure the inputs off the balance sheet

Horizon (`parseHorizon`)

Parses `goals.horizon` for a phase and a midpoint. `/drawdown|decumulat|retire/` → `drawdown` phase, else `accumulation`. Two numbers → their average as years; one → that number; none → default 7 years.

Funding goal → required return and/or income need (`parseFunding`)

Handles three headline shapes. An income unit (`"…/yr"`) or a `"$X m/yr"` match yields an **annual income draw**. A `"to $X m"` match yields a **terminal value target**. The required return is then a compound-growth solve:

```
requiredReturn % = ( (terminalTarget / baseValue) ^ (1 / years) − 1 ) × 100
```

- `baseValue` = explicit `funding.current` for value goals, else AUM.
- `years` = an explicit "by 20XX" minus 2026 (the env year), else the horizon midpoint.
- Only computed when `terminalTarget > baseValue` (no negative required returns).

Liabilities → near-term bullets vs serviceable debt (`parseLiabilities`)

Each liability is classified by name/note keywords and summed, all as a **% of AUM**:

Quantity	Trigger	Feeds
`nearBillPct`	`near:true` or tax / due / bridge / bill / payable / Q1–4 / settlement	Preservation reserve
`servicePct`	mortgage / loan / line / credit / leverage / margin / borrow → `amount × rate` (rate default 6%)	Income (debt servicing)
`debtLoadPct`	every liability's amount (total leverage)	Income & Preservation matching/ballast

Willingness — stated and revealed (the two-way read)

Stated (`statedWillingness`) parses the free-text risk string to a 0–1 scalar: aggressive/max → 0.95, growth → 0.80, moderate/balanced → 0.50, conservative/income/preservation → 0.30, unparseable → `null` .

Revealed (`revealedWillingness`) reads it off the *current book*, three signals blended:

```
R = share of book in risk-on assets (Equity + Alternatives)         clamp 0–1
D = 1 − min(1, defensiveShare / 0.55)   defensiveShare = Cash+FixedIncome+Commodity
C = min(1, (largestGenuineSingleName% / 100) / 0.25)   concentration appetite

revealed willingness = 0.5·R + 0.3·D + 0.2·C            (clamped 0–1)
```

The blended willingness is `0.5·stated + 0.5·revealed` when a risk profile is stated, and purely `revealed` when it isn't (Mitch, Fotis). The "largest genuine single name" excludes index cores and fundless sleeves (no ticker / sector "Broad").

4.2 Step 2 — build the three raw bucket scores

Every bucket starts at a small `base = 8` and accumulates named, inspectable terms. Let `caution = 1 − willingness` , `willingCoef` = 30 when a required-return signal exists, else 45 (lean harder on willingness when there is no written goal), and `concExcess = max(0, topName% − 15)` .

Bucket	Raw score = sum of these terms
Preservation	`8 + 1.0 × nearBillPct + 0.3 × debtLoadPct (ballast vs leverage) + 1.0 × concExcess (single-name > 15%) + 15 if drawdown phase + 3 × max(0, 5 − horizonYears) + 28 × caution (temperament ballast)`
Income	`8 + 8 × incomeNeedPct (capitalises the draw) + 8 × servicePct (debt service) + 0.6 × debtLoadPct (income-match the debt) + 15 if drawdown phase + 18 × caution (stable-cash preference)`
Growth	`8 + 5 × requiredReturn% + 3 × max(0, horizonYears − 6) + willingCoef × willingness`

> **Willingness is bidirectional.** A low appetite must *add ballast*, not merely shrink Growth — otherwise a client with no liabilities and no income need comes out growth-heavy no matter how cautious they are (Income/Preservation would only ever get weight from needs). The `caution = 1 − willingness` terms on Preservation (×28) and Income (×18) fix this.

4.3 Step 3 — normalise to exactly 100

The three raw scores are normalised with **largest-remainder rounding** (`normalizeTo100`) so the vector always sums to *exactly* 100 with integer parts — floor each scaled value, then hand the leftover points to the largest fractional remainders. This avoids the "99% / 101%" artefacts of naive rounding.

4.4 Provenance — source & confidence

Condition	`source`	`confidence`
A funding goal yields a required return	stated-goal	0.90

No goal, but a risk profile is stated	stated-risk	0.70
Nothing on file — read purely off the book	revealed	0.55

A human-readable `drivers[]` list is assembled (e.g. "Needs ~6.3%/yr over 9y to hit the funding goal → Growth"; "25.8% single-name concentration → Preservation") and surfaced verbatim in the Advisor Book's "How were these goals derived" panel. A `client.goalOverride` pins a manual vector and short-circuits the whole model (source "override", confidence 1).

4.5 Folding the *current* book into the same three buckets

To draw the "now vs goal" bar, the live book is folded into the same three buckets by **role** (`currentBuckets`), so it reconciles with the derived goal:

Asset class	Folds into
Equity, Alternatives (incl. crypto, venture)	Growth
Fixed Income, Credit, Real Assets	Income
Cash, Commodity (gold)	Preservation
Structured notes	by *purpose*: buffer/protect/capital → Preservation; autocall/coupon/income → Income; else Growth

There is no separate "Structured" or "Liquidity" goal bucket; cash and gold are ballast (Preservation), and structured notes are classified by what they do.

5 · The unified idea→client fit engine MAPPING.JS

> There is **one** fit scorer: `window.MAPPING.scoreIdeaForClient(idea, client)`. Everything funnels through it — Today's Focus flagging, the Advisor-Book top-3, the Draft-a-view preview, and the Solutions Views client lists. In `scanner.js`, `ideaFit` / `clientsForIdea` / `matchedIdeas` are **thin facades** that call it and keep their legacy output keys. So the saved-View client list and the draft-preview list can never disagree — same algorithm, same gate.

5.1 The shape of the calculation

Client-fit is a **gated, goal-aligned weighted sum of four transparent axes**, each scored 0–100 with a plain-English note. In one line:

```
Final Client-Fit = Tradability × GoalAlignment × Σ( axisᵢ.score × weightᵢ )

where  Σ weightᵢ = 1.00   (so the bracketed sum stays 0–100)
       Tradability    ∈ {0, 1}      binary MiFID gate  (§7.1)
       GoalAlignment  ∈ [0.85, 1.15] multiplier         (§7.2)
```

5.2 The four axes and their flat weights

Weights are **flat** (fixed per axis), held in `PARAMS.weights`. They are the original five-axis weights with House-view fit (0.15) removed and the remaining four rescaled by `1 / 0.85` so they keep their relative balance and sum to exactly 1.00. The **exact fractions** are used in the math; the UI shows rounded values.

Axis	What it measures	Exact weight	Shown
Affinity fit `holdings`	Recency-weighted sector affinity – over-limit penalty	0.25 / 0.85 = 0.2941…	0.29
Mandate & risk `mandate`	0.6·RiskSuitability + 0.4·IntentFit	0.25 / 0.85 = 0.2941…	0.29
Gap fit `gap`	Headroom toward the client's goal-bucket target	0.20 / 0.85 = 0.2353…	0.24
Concentration within sector `concSector`	(1 – Herfindahl)×100, inverted for fit	0.15 / 0.85 = 0.1765…	0.18
Total		**1.0000**	**≈1.00**

5.3 Tiering and gates

Output	Rule
Tier	`fit ≥ 66` → **Strong** · `≥ 48` → **Good** · else **Marginal**
`applies` (Views / draft)	tradable **and** aligned weighted sum `≥ 45` (`applyMin`)
`flagClients` (Today's Focus)	`fit ≥ 50` (`flagMin`), top 6 (`flagMax`)

`why`	the suppression reason when gated, else the highest-*contribution* axis's note

Suppressed (non-tradable) clients have `fit = 0`, so they fall out of every flag/apply list, but the UI **surfaces them separately** with the MiFID reason rather than silently dropping them (`suppressedClients(idea)`).

5.4 Idea descriptors — explicit, with derived fallbacks

The axes read a few derived descriptors when an idea doesn't carry them explicitly:

- **`naturalExpression`** = `structures[0]` if absent. The expression the tradability gate tests.
- **`goalTypeOf`** → appreciation | yield | protection. From the bucket, refined for Structured ideas by scanning the structures/title text.
- **`riskProfileOf`** → `{vol, beta, structured}`. `structured` from asset-class/structures; `beta` from the sector (HIGH_BETA = Technology, Crypto, Materials, Energy, Industrials, Consumer; LOW_BETA = Utilities, Gold, Rates, Credit, Infrastructure, Real Estate, FX; else moderate); `vol` from bucket + beta + structured.

6 · The four fit axes, in full

All constants live in `PARAMS`. The axes operate on **sector-level** signals (sector exposure, the 24-month history, in-sector holdings) plus the client's parsed mandate. The client's **mandate class** — growth / income / preservation — is derived from the parsed risk string (`mandateClass`), and the **strategic sector peg** is a single source of truth (`PARAMS.affinity.comfortByBucket = {Growth:25, Income:15, Preservation:12}` — keyed by the *asset's* goal bucket, with optional per-sector overrides — read by `sectorPeg(client, sector)`).

6.1 Gap fit — bucket-aware (weight 0.24)

Does the idea fill a *goal bucket* the client is under on? It pegs the idea's bucket to the client's own **derived goal vector**, not the sector ceiling:

```
Gap fit = max( 0, (bucketTarget − bucketCurrent) / bucketTarget × 100 )

bucketTarget  = goalsFor(client)[idea.bucket]      (the derived goal, §4)
bucketCurrent = currentBuckets(client)[idea.bucket] (the live book, §4.5)
```

So a Preservation idea (gold) scores **0 once the client's Preservation bucket is at/over target**, rather than being judged against a sector ceiling. Worked (Preservation goal 35%): current 10 → 71, 25 → 29, 35 → 0. **Fallback:** if the client has no goal target for that bucket, it reverts to sector headroom vs the mandate comfort peg, `max(0, (peg − sectorCurrent)/peg × 100)`.

Live example. Gold × Amar: his Preservation bucket sits at 35% against a 35% derived goal → Gap fit **0** (no headroom), correctly dropping his overall fit from a misleading "Strong" to "Good." Clients under their Preservation target score high on the same idea.

6.2 Affinity fit (weight 0.29)

```
Affinity fit = max( 0, ThematicAffinity − ConcentrationPenalty )
```

A · Thematic Affinity (0–100) — a recency-weighted percentile of the current sector allocation within the 24-month history. With decay `λ = 0.94` (month `t` weight = 0.94^t, `t=0` most recent):

$$\text{ThematicAffinity} = \frac{\sum_t w_t \cdot [\text{history}_t \le \text{current}]}{\sum_t w_t} \times 100, \quad w_t = 0.94^t$$

i.e. "how high in its own trailing-24-month range is this book's current allocation, weighting recent months more." Edge cases: no current exposure → 0; holds the sector but no history on file → treated as steady → ~100.

B · Concentration Penalty (subtracted) — how far the current allocation overshoots the mandate's sector comfort peg:

```
Penalty = max( 0, (current − sectorPeg) ) × 10,   capped at 100
```

Affinity is sector-based; Gap fit is now bucket-based, so the two no longer share one peg — only the Affinity penalty reads `sectorPeg`.

6.3 Mandate & risk (weight 0.29)

```
Mandate & Risk = 0.6 × RiskSuitability + 0.4 × IntentFit
```

(Tradability is *not* on this axis any more — it is a global gate, §7.1, so keeping it here would double-count.)

Risk Suitability (0–100 + reason) — a deterministic matrix of the idea's `{vol, beta, structured}` against the mandate. Highlights:

Mandate	High-beta/high-vol	Moderate	Low-vol	Capital-protected
growth	92	72	55	—
income	45	82	90	—
preservation	22	50	80	92

Intent Fit (0–100 + reason) — a matrix of the idea's goal type against the mandate's goal (`INTENT_FIT[mandate][goalType]`):

Mandate ↓ / Goal type →	appreciation	yield	protection
growth	90	55	45
income	60	90	65
preservation	30	68	92

Worked. Income client + a low-vol dividend equity (`vol:low`, `goalType:appreciation`): Risk Suitability 90, Intent Fit 60 → `0.6·90 + 0.4·60` = **78**.

6.4 Concentration within sector (weight 0.18)

A Herfindahl diversification score of the book's holdings *inside the idea's sector*:

```
HHI = Σ (wᵢ)²    over in-sector holdings, weights normalised to sum to 1
raw = (1 − HHI) × 100     concentrated → 0, diversified → 100
```

Worked: one name → HHI 1.0 → raw 0; five equal names → HHI 0.20 → raw 80.

> **Inverted for fit by default.** A concentrated sector position *needs* a new name, so it should fit **more**: `fitContribution = invertForFit ? (100 − raw) : raw`, controlled by the single flippable line `PARAMS.concWithinSector.invertForFit` (default `true`). The breakdown shows both the raw diversification score and the fit contribution. No in-sector holdings → neutral 50.

Live example. Fable's six spread Technology names → HHI ≈ 0.19, diversification 81, inverted fit contribution **19** — already diversified within tech, so a new tech name adds little.

6.5 Why House-view fit is *not* a fit axis

House-view fit was removed from client-fit because it is an **idea-level property**, not a per-client signal, and it double-counted holdings already captured by Affinity and Gap fit. It now lives in the **conviction** score instead (§8), as a pillar of the ex-earnings model. The on-theme / off-theme tag still shows on every tile.

7 · The gate and the multiplier

7.1 The global tradability gate (binary, MiFID)

Computed **once** per idea×client, *before* the weighted sum, and multiplied across the whole score. Not an axis — it used to be one term on Mandate & Risk, but that let a non-tradable idea still score up to ~75; moving it to a global gate means it fully suppresses the idea.

```
tradability(idea, client) =
   0   if  client.classification == "Retail"  AND  isOtcOption(naturalExpression)
   1   otherwise        (Professional, or a non-complex / structured-note expression)
```

A `0` ⇒ `fit = 0`, `suppressed = true`, `applies = false`, and the reason ("Not tradable — Retail classification doesn't permit … (OTC derivative)…") is surfaced in the UI. Because the four weights already sum to 1.00, the gate only ever passes the bracket through (×1) or zeroes it (×0); nothing is re-normalised.

Live example. Aurora (Retail) + a zero-cost collar (OTC) → tradability 0 → fit 0, suppressed with the MiFID reason. A Professional, or a structured-note expression, → 1 and the score passes through.

7.2 The goal-alignment multiplier (0.85–1.15)

A goals-aware nudge over the bracketed score (`goals.js::goalAlignment`). It reads the client's derived 3-bucket goal and boosts an idea that fills a bucket the client is *under* on, trims one that piles into a bucket already *over*:

```
b   = goal bucket the idea serves (goalBucketOfIdea)
gap = goalsFor(client)[b] − currentBuckets(client)[b]      (+ = under-allocated)
mult = clamp( 1 + gap / 200,  0.85,  1.15 )
```

This is the signal that makes Today's Focus, Views and the Advisor Book all converge on the clients whose *goals* an idea actually serves. (Gap fit, §6.1, captures the same intuition as an axis; the multiplier applies it once more across the whole score, deliberately bounded so it nudges rather than dominates.)

7.3 Putting it together — `scoreIdeaForClient`

```
bracketFit  = round( Σ axisᵢ.score × weightᵢ )      # four axes, Σweight = 1.00 → 0–100
alignedFit  = bracketFit × goalAlignment.mult       # 0.85–1.15
fit         = tradable ? round(alignedFit) : 0      # binary MiFID gate
tier        = fit ≥ 66 Strong · ≥ 48 Good · else Marginal
```

The returned object is a superset every call site consumes: `{ fit, tier, why, axes[4], bracketFit, tradable, suppressed, tradabilityReason, naturalExpression, goalAlignment, applies, score, reason, gap, secExp, acExp }`. Each `axes[]` row is `{ key, label, weight, score, contribution, note }` — the per-axis breakdown that opens on click, so the reasoning is never a black box.

8 · Conviction scoring — two rubrics BUILD_TODAY_FOCUS.PY

Conviction is the idea's *standalone* quality, computed by the generator (not the browser) from the research payload. There are **two rubrics, chosen by** `idea.kind`. Each pillar carries its own `max` and a data-quality tag `dq ∈ {sourced, estimated, unverified}`. The generator stamps labels/maxes, sums the raw score, scales to /100 and bands it.

8.1 Earnings ideas — the print rubric (/8)

Four pillars, each **0–2**:

Pillar	0–2 criterion
Asymmetry signal	implied straddle move ÷ avg absolute realised move over the trailing **4 prints** — materially <1 (market underpricing) → 2 · near fair → 1 · implied rich → 0
Sell-side consensus	majority buy / upside to mean PT / positive last-30-day revisions — all three → 2 · partial → 1 · none/contradicts → 0
Catalyst clarity	pre-print: a dated, unpriced catalyst; post-print: a reaction disproportionate to print quality
Positioning & sentiment	short interest >10% float OR an extreme positioning setup

The **banding** governs the label directly (not a simple /100 cut):

```
High           raw ≥ 6, no zero pillar, all inputs sourced
High – data gap same, but ≥1 estimated input
Medium         raw 4–5, OR any zero pillar, OR consensus unverified
Low            raw < 4, OR two-plus zero pillars   (excluded)
```

8.2 Ex-earnings ideas — the seven-pillar model (/15)

Pillar	Max	What it scores
Gap / asymmetry	3	thesis expected move (entry→target) ÷ the name's normal move: ≥2.0→3 · 1.5–2.0→2 · 1.0–1.5→1 · <1.0→0
Catalyst	2	a dated, hard reason it moves soon (earnings, FOMC, CPI, ECB)
Consensus + confirmation	2	sell-side alignment *or* a defensible variant, AND an independent signal (breakout/flow/skew)
Fundamental thesis	2	two 0/1 checks: driver specificity (named, quantified) + variant view (computable gap vs consensus)
House-view fit	2	core desk theme / direct → 2 · on-theme / adjacent → 1 · off-theme / against → 0
Positioning	2	confirmation/veto: SI >10% float, extreme CoT, fund flows + RSI sentiment read
Technical	2	confirmation/veto: 50/100/200-day MA alignment, Bollinger entry, Fibonacci off the 6-mo swing

Total	**15**	

```
score = raw ÷ 15 × 100      High ≥ 75 · Medium ≥ 55 · Watch ≥ 0
```

> **The view drives; technicals confirm — roughly 70/30.** Fundamental thesis + house view + the thesis-bearing pillars (Asymmetry, Catalyst, Consensus) carry **11/15 ≈ 73%**; Positioning + Technical carry **4/15 ≈ 27%** and are framed as *confirmation / veto, not drivers* — they can support or kill a view but can't manufacture conviction. RSI lives only in Positioning; moving averages only in Technical — no overlap.

8.3 The data-quality cap (global modifier)

If *any* pillar's core input is `estimated` or `unverified` (`capped = true`), the ex-earnings label cannot exceed **Medium**; the earnings model expresses the same idea through the "High — data gap" label. The **Technical** pillar is now sourced live from TradingView by `tv_technicals.py` (§9.2), but the straddle and short-interest inputs remain `estimated`, so the cap still binds most ideas — an honest ceiling that lifts pillar by pillar as sourced feeds are wired in, not a hidden one.

8.4 `changeMyMind` — stated but not scored

Every idea — earnings and ex-earnings — carries a `changeMyMind` field: the condition that would invalidate the view. It is surfaced on the conviction tile but **deliberately not scored**, since a discrete kill-switch would bias the score toward dated catalyst trades over strategic holds.

9 · Where ideas come from

Ideas enter the system through **three distinct channels**, all tagged into the same four-dimension schema (type / asset class / sector / bucket) so the downstream engines treat them identically.

9.1 Channel A — the standing Solutions Views board

The ~17 seed ideas in `data.js` are the desk's **standing themes**, synthesised around a J.P. Morgan Private Bank mid-year-outlook positioning (AI & productivity, power & infrastructure, extending duration, broadening beyond mega-caps, real assets, resilience/protection, gold & currency, healthcare, structured outcomes). The copy is original; each idea is hand-authored once with its `thesis`, `conviction` and `structures`, then **everything client-facing is derived** — which clients it applies to is computed, never typed.

9.2 Channel B — the daily market sweep (the Idea Feed's board)

The feed's board is **schedule-ready**: it is generated from a data file, never hand-coded into the app. A refresh is one pipeline run:

Step	File	What happens
1 · Sweep	`today_focus.json`	A Claude Code session verifies the market across five workstreams (§14.1), applies the **≥2-source rule**, tags every fact `sourced` / `estimated`, authors each idea's pillar scores + `dq` tags as a first pass, and overwrites the raw payload — ideas, the market brief, and a desk note recording what changed and why.
2 · Technicals	`tv_technicals.py`	Rewrites each idea's **Technical** pillar from TradingView's public scanner (SMA 50/100/200 alignment, Bollinger entry, RSI), *direction-aware* — a rising trend confirms a long and vetoes a short; range/carry ideas are scored on tape-calmness instead — and flips that pillar's `dq` to `sourced`. Positioning gets the RSI reading as colour but keeps its authored `dq`: short interest / CoT aren't in the feed, and flipping the tag wholesale would be dishonest. Unresolvable symbols are reported and left `estimated` — never silently skipped.
3 · Charts	`fetch_chart_series.py`	Fills each idea's chart block with **real Yahoo Finance daily closes** plus a provenance stamp (`seriesSource`, `seriesAsOf`). The build **strips any chart without a sourced series** — a chart that claims to show market moves must *be* market data.
4 · Build	`build_today_focus.py`	Validates everything (below), applies the conviction rubric (§8), runs the tactical gate and the date ledger, and writes `today_focus.js` (`window.TODAY_FOCUS`) — auto-generated, never edited by hand.
5 · Ship	`git commit + push`	The site deploys from the remote; the client mapping is computed in the browser at render time, so flagged clients always reflect the current book with no app edits.

What every sweep must cover

The sweep is a **whole-market** pass, not a US-equity screen, and the validator warns when coverage slips:

Requirement	Rule
Asset classes	Equities, rates, credit, FX, commodities (+ structured where it fits); regions: US, Europe, UK, Asia. **FX and rates must appear in every run** — they are the most often missed.
Earnings stances	The earnings book must carry **both**: `pre-position` (into a future print — report date must be ≥ asOf, kept future-tense) and `post-print` (a reaction to a landed print — report date < asOf **and** the actual result stated as a verified fact, `resultSource: "sourced"`; fabricated results are a validator defect).
Tactical trades	At least one short-dated option trade with a concrete construction and a `levels` block — tenor, entry, target (early unwind), stop — primarily in FX.
Dislocation scan	Large-cap drawdowns are candidates, not noise: any widely-held name >~15% off its highs is scanned and either actioned or logged with the reason it wasn't.
Expression	**Derivatives-first:** every idea leads with an options / structured / OTC construction (`preferredExpression`); direct lines are listed as alternates. The MiFID taxonomy (§3.6) then decides who can trade what.

The source rules — and citation integrity

A fact is stated only if **two independent sources** agree; single-sourced figures are tagged `estimated`; pending events stay future-tense. On top of the broad tape (Reuters, Bloomberg, CNBC, official releases), the desk reads **named authors** in two tiers:

Tier	Who	When
Core	Citrini Research (thematic) · Serenity · TSCS · Brent Donnelly, am/FX (FX) · Cem Karsan (vol / dealer positioning) · Joseph Wang, Fed Guy (rates & Fed plumbing) · SemiAnalysis (AI capex / semis)	Every sweep, alongside the tape
Conditional	The Transcript (earnings) · Benn Eifert, Moontower, Tier1 Alpha (options & vol) · Fabricated Knowledge (semis) · Rory Johnston, Doomberg (commodities) · HighYield Harry (credit) · The Market Ear, Brad Setser (flows) · Andy Constan, Joseph Politano (macro data)	When an idea touches their domain — the natural second independent source

```
CITATION INTEGRITY (hard rule, validator-enforced):
a source of kind "view" MUST carry a `basis` — what was read, when, and what was
taken from it. No basis → the build rejects the citation.
Nothing readable found → the idea carries ownView: true and the card says
"Own view — no outside research leaned on".
Fresh commentary that merely RESTATES an unchanged thesis is not a reason to
cite it or re-date the idea — the ledger keeps the dates honest.
```

An author's read informs conviction; it never bypasses the rubric or the client-fit mapping. And attribution stays disciplined the other way too: moves are attributed to *named, verified* drivers, never to the scariest headline nearby.

The tactical trigger gate

Event-driven option trades must not sit on the board permanently. Any idea carrying a `levels` block must also declare:

```
trigger    the stated condition that justifies the trade NOW
                        (e.g. "a confirmed daily close above 1.17")
triggered  boolean — true only when that condition is actually met today
```

Only **triggered** tacticals ship to the app; untriggered ones stay in the payload and are reported as *held back* on every run — an auditable record of discipline. The board has carried the EUR/USD and GBP/USD upside trades as held-back for weeks because spot sat below their stated levels; "1.3503 is not 1.355" is the system working, not the idea failing.

Honest dates — the fingerprint ledger

Every card's "posted Nd ago" reads the idea's own `postedAt`, and a committed ledger (`today_focus.ledger.json`) keeps those dates honest. The build fingerprints each idea's **argument** — tradeStatement, headline, thesis, changeMyMind, direction, intent, structures, fact texts, pillar scores/notes, levels, the earnings result — deliberately excluding chart series, indicator timestamps and computed totals, so a pure price refresh doesn't count as change:

```
id unseen            → postedAt = updatedAt = asOf        (a new idea)
fingerprint same     → both dates carry                   (a no-op re-commit moves nothing)
fingerprint changed  → postedAt carries, updatedAt = asOf (the "updated Nd ago" chip)
updatedAt > 10 days  → the build WARNS: re-ground the commentary or retire the idea
```

The shape of one swept idea

Each idea carries: identity (`id, name, ticker, sector, assetClass, bucket, goalType, riskProfile, direction`); the argument (`headline, thesis`, a required one-sentence `tradeStatement`, the `variant` street/us/gap triplet that feeds the thesis pillar, `changeMyMind`); its provenance (`themeId` or `offThemeWhy`, optional `ownView`); its expression (`preferredExpression, structures[], naturalExpression` — the string the tradability gate tests); the authored `conviction.pillars[]` (each `{key, score, dq, note}`); an `earnings` block (stance-dependent: implied-vs-historical move pre-print, the verified result post-print) or an ex-earnings `macro` block; optional `tactical / trigger / triggered / levels` and a `chart` block; and the evidence — tagged `facts[]` and typed `sources[]`.

Validation — warn, never silently pass

- ≥2 sources per idea · every fact tagged · `tradeStatement` present (FX must name the long and short legs and spot-vs-carry) · view-citations must carry a basis.
- Stance-aware date checks (a forward event must never be described as already happened; a reaction idea needs a verified result).
- Coverage checks (FX, rates, both stances, tacticals) · tactical-gate contract · chart integrity (unsourced series stripped) · pillar count/keys/max checks · intent defaulting.

9.3 Channel C — Draft a view (advisor-authored, on the fly)

The advisor types a one-line thesis; `draftFromThesis()` matches it against a keyword table (`DRAFT_RULES`) to suggest a theme, sector, asset class, bucket and expressions:

Keywords (any match)	→ Theme	Sector / Class / Bucket
ai, semiconductor, chip, gpu, compute, hbm, nvidia, datacenter, software	AI & Productivity	Technology / Equity / Growth
power, grid, electric, utility, nuclear, smr	Power & Infra	Utilities / Equity / Income

bond, duration, yield, treasury, rates, fixed income	Core Fixed Income	Rates / Fixed Income / Income
gold, bullion, debasement	Gold & Currency	Gold / Commodity / Preservation
protect, hedge, downside, collar, buffer	Resilience & Protection	Broad / Multi-Asset / Preservation
health, glp, pharma, medtech, biotech, device	Healthcare Innovation	Healthcare / Equity / Growth
coupon, autocall, structured note, income note	Structured Outcomes	Broad / Structured
copper, materials, mining, rare earth, lithium	(off-theme)	Materials / Equity / Growth
energy, oil, gas, crude, brent	(off-theme)	Energy / Equity / Income
crypto, bitcoin, digital asset	(off-theme)	Crypto / Alternatives

The synthesised idea is then immediately scored against the live book with `MAPPING.flagClients(synth, {min: 45, max: 5})`, so the advisor sees **candidate client books with live fit scores before saving**. On save it is appended to `userData.ideas` and persisted to `localStorage`, joining the Views board exactly like a seed idea. No keyword match falls back to a generic Equity / Broad / Growth idea.

9.4 Channel D — Solutions curation in the shell

The redesigned shell adds a fourth entry point (§2.4): the Solutions view's **composer**. Advisor drafts join the feed under the author's own name, are *not* desk-scored (the rubric only scores the sweep), and map to clients through the same live fit engine as everything else. The same view holds the **approve / discard gate** over the sweep's output — so a human desk signs off on what the automated sweep proposes before an advisor ever sees it. Curation state persists in `localStorage`; the engines are untouched.

10 · The portfolio scanner — 11 rules SCANNER.JS

Separate from the fit engine, `scanBook(client)` runs the book *the other direction*: portfolio → ideas. It is eleven hand-coded, severity-ranked rules that read the positions and surface actions. It powers the "See more ideas" asset-class list and the Next Best Action — it is **not** a per-client fit score. Each finding carries a severity (1–3), title, rationale, suggested structures, bucket, and a Retail-appropriateness flag.

#	Rule	Trigger	Sev
1	Protect concentration	single-name Equity/Alt ≥ 15% (not index/crypto)	2–3
2	Loss harvest	equity-like position ≤ −10%	2
3	Bond swap	Fixed Income ≤ −8% (rates, not credit)	2
4	Rehabilitate crypto	Crypto position ≤ −20%	3
5	Overwrite a winner	Equity ≥ +50% & <15% weight	1

6	Hedge FX mismatch	non-base, non-cash currency ≥ 40% of book	2
7	Put cash to work	Cash ≥ 8%	1
8	Liability & cashflow planning	any liabilities present	3
9	Close the protection gap	Preservation target – current ≥ 6pts	2
10	Lift income	Income target – current ≥ 8pts	1
11	Diversify a sector	any sector ≥ 30% (excl. Cash/Rates/Credit/Broad)	1

Rules 9 and 10 reference the *derived goal* (§4), so the scanner's protection/income gaps are consistent with the goal-inference engine. After generating findings, the scanner **re-normalises appropriateness** from the actual structures: a finding is only Retail-blocked when *every* way to express it is OTC — if a structured-note or non-complex route exists, Retail can take that instead. Findings are sorted by descending severity; the top one (else the top matched View idea) becomes the **Next Best Action**.

11 · Supporting analytics — charts, Morgan & the email engine CHARTS.JS · MORGAN.JS · EMAIL.JS

11.1 Allocation analytics (`charts.js`)

- **Donut / legend** — pure-SVG allocation rings from a split, in the house palette.
- `bucketAlloc(split)` — folds an asset-class split into the three goal buckets (delegates to `goals.js` so every consumer shares one classifier).
- `goalTargetBar3(target, current)` — the "now vs goal" bar: the *bar* is the book's current weight in each bucket, the *notch* marks the inferred goal, and a label reads "Now X% · target Y% · ±Δ" (on plan if |Δ| < 4).
- `targetDistance(buckets, target)` = $\Sigma\ |\text{bucket}_k - \text{target}_k|$ — the L1 distance from plan — used to say a proposed change moves the book *closer to* or *further from* target.
- `fundingBar(f)` — funding-goal progress, `min(100, current/target × 100)` with an On-track / Slightly-behind / Behind pill.

11.2 Morgan — "Ask your book" (`morgan.js` · `methodology.js`)

The shell's right-rail assistant answers questions about clients, ideas and risk from the live engines: per-client fit (with the axis breakdown), conviction and its pillars, comfort limits, and the goal derivation. Its knowledge base is `window.METHODOLOGY`, which **reads every constant live from the engines' PARAMS objects** — so the in-app description of the methodology can never drift from the code that runs it, and the assistant never invents a number.

11.3 The email engine (`email.js`)

Any idea × client becomes a personalised letter: a real-holding hook (naming a position and its P&L), the desk view in client language, a concrete implementation block (structure, tenor, terms — the Solutions-signed-off text where one exists), a tax-swap / loss-harvest / cash-redeploy / FX-sizing action where the scanner flags one, a balanced risk paragraph, and a disclosure. One shared code path serves every surface, so the copy can't drift between the card, the toolkit and the drafted email.

12 · Worked end-to-end example — gold for Amar

To see the whole machine run on one idea×client pair, take **"Gold as a debasement hedge"** (Commodity / Gold / Preservation, natural expression "Physical / ETC") scored for **Amar** (Professional; ~35% of his book already in Preservation-role assets via gold + cash).

1. **Goal inference (§4)** derives Amar's three-bucket goal from his real-asset book, multi-decade horizon and growth-tilt willingness — placing his Preservation target at ~35%.
2. **Gap fit (§6.1)**: Preservation bucket current ≈ 35% vs goal 35% → `max(0,(35-35)/35)` = **0**. No headroom — gold doesn't fill a gap he has.
3. **Affinity fit (§6.2)**: his Gold allocation sits high in its own 24-month range (a "ramp" shape) → high thematic affinity, but possibly an over-peg penalty; net moderate-high.

4. **Mandate & risk (§6.3)**: gold's `{vol: moderate, beta: low}` against a growth mandate → middling Risk Suitability; protection goal type vs growth mandate → Intent Fit 45.
5. **Concentration within sector (§6.4)**: he holds gold + GDX miners → some diversification, modest inverted fit contribution.
6. **Bracketed sum** of the four weighted axes → a 0–100 `bracketFit`.
7. **Goal-alignment multiplier (§7.2)**: Preservation gap ≈ 0 → multiplier ≈ 1.0 (no boost).
8. **Tradability gate (§7.1)**: Professional + physical gold (non-complex) → **1** → passes.

The Gap-fit zero is what correctly demotes gold for Amar from a misleading "Strong" to a "Good" — he already *has* his protection. The exact same idea scores high for a client *under* their Preservation target (Scott, Fotis, Aurora). That is the system working as intended: **the fit follows the client's goals and book, automatically.**

13 · Calibration, limitations & honesty

The project is deliberately candid about what is and isn't real:

- **The `PARAMS` numbers are reasoned, not fit to outcomes.** Every coefficient (the affinity λ, the willingness weights, the axis weights, the intent matrix) is a calibration choice kept in one visible place. The in-browser harness — load the app, score against `window.SEED` — is how they were sanity-checked and where they'd be recalibrated.
- **The 24-month sector history is synthetic seed data**, clearly labelled, swappable for a real monthly series with no other change.
- **The data-quality cap still binds most ideas.** The Technical pillar is now sourced live (TradingView, §9.2), but straddle and short-interest inputs remain estimated, so conviction tops out at Medium / "High — data gap" until those feeds are wired — an honest reflection of the data on hand, not a hidden ceiling.
- **What still doesn't enter the fit:** AUM, currency-hedging depth, liquidity and liability funding (the scanner handles some of this separately). `riskProfile` / `goalType` are derived sensibly but per-idea curation would sharpen them.
- **The concentration fit-direction is a product decision** (`invertForFit`) — one flippable line, surfaced for confirmation rather than buried.

> **The through-line.** Every number on screen — a goal vector, a client-fit score, a conviction band — is a pure function of inputs you can inspect, with a written reason attached. Nothing is hand-mapped, nothing drifts, and every claim can be opened up and traced to the formula that produced it. That transparency *is* the methodology.

14 · The daily operating loop — how a refresh actually runs

Sections 8–9 describe the machinery; this is the **morning routine** that drives it. There is no scheduler wired up — the cadence is a convention, and each run is an on-demand Claude Code session.

14.1 Verify the market — five workstreams

Before a single idea is written, the sweep establishes the verified state of the market, in parallel, each fact needing two independent sources:

Workstream	What it must establish
Fed / rates / macro	The policy decision path, current pricing for the next meeting, the curve (2s/10s/30s), the last jobs and inflation prints, and the dated calendar ahead (CPI, PPI, minutes, symposia).
Energy / gold / commodities	Crude levels and path, the live geopolitical mechanism (not just headlines), OPEC decisions, gold with its flow drivers (central-bank purchases, ETF flows), and downstream confirmation (cracks, gas).
FX / central banks	The majors and their central-bank vectors, any intervention (confirmed by whom), and the desk's tactical trigger levels checked against verified spot.
US equities / earnings	Index levels and path, the earnings calendar two weeks out with consensus and implied moves where findable, recent prints with their *reactions*, and the standing dislocation scan (>15% off highs).
Global + named authors	Asia and Europe levels and stories, plus the author list of §9.2 — locating and actually reading what is readable, recording what was not found.

14.2 Re-underwrite the board

Every existing idea is re-graded against the fresh facts, and each lands in one of five outcomes — all recorded in the sweep's desk note:

Outcome	What it means	Mechanics
Retire	Played out, thesis overtaken, or stale beyond re-grounding	Delete from the payload; the ledger forgets it; the note records why
Re-ground	The thesis holds; the argument is refreshed to current levels and tense	Same `id` — postedAt carries, updatedAt bumps, the "updated" chip shows
Flip	An earnings idea's print has landed	Same `id`, stance pre-position → post-print, the verified result added
Hold back	A tactical whose trigger is still unmet	Stays in the payload, `triggered:false`, reported not shipped
Add	A new idea earns its place from the screen	New `id`, full contract (§9.2), first-pass pillars authored honestly

> **Voice discipline.** The board is an *overnight refresh*: commentary is written in the present tense off the current setup, and the past appears only where a landed result is itself the fact (a post-print result block, a banked trade's roll). Trade accounting — what paid, what lost — lives in the desk note and the git history, not draped over every thesis.

14.3 The desk note — the audit trail

Alongside the brief, every sweep writes a `note` that archives the run: the verified data with its discrepancies logged (and discarded outliers named), what changed on the board and why, what the gate held back, the dislocations scanned-but-not-added with reasons, which named authors were actually read this run (and which could not be found — never cited), the attribution line, and the honest staleness stamps for anything not re-run. Prior notes live in the git history rather than accumulating inline — the ledger, not the note, is the date authority.

14.4 Ship it

```
python tv_technicals.py        # Technical pillars ← TradingView, sourced
python fetch_chart_series.py   # chart series ← Yahoo closes, provenance-stamped
python build_today_focus.py    # validate · score · gate · ledger → today_focus.js
git commit -m "chore: market sweep refresh YYYY-MM-DD (one-line desk summary)"
git push                       # the site deploys from the remote — a local commit is
invisible
```

Fix every validator warning rather than suppressing it; the commit message convention makes the git log double as the desk's sweep journal.

15 · Design system & deployment

15.1 Two coordinated design systems

	The shell (`openbook.css`)	The legacy pages (`styles.css`)
Feel	Editorial feed — kickers, colour strip-bars, conviction rings, soft card shadows	Printed research note — hairline rules, tag chips, no shadows
Ground / ink	`#FBFAF7` / `#17150F`, white surfaces	warm paper `#FBF8F2` / espresso `#29211A`
Accents	bronze `#996F3D` · light bronze `#C7A97A` · gold `#EBC98D` (the dark-mode accent) · greens `#2FBF71` / `#2C7A4B` · brick `#9C4A3A`	bronze `#9A7B4F` · umber `#5A3E2B` · green `#3F6B4E` · muted red `#8C3F3F`
Type	**Source Serif 4** (display) + **Archivo** (UI), Google Fonts	Georgia-stack serif + letter-spaced Helvetica small-caps
Dark mode	`[data-theme="dark"]` on `<html>`; toggle in the header	Its own dark mapping, driven by the same key so embeds match

Theme mechanics: one `localStorage` key (`ob-theme`) shared by all pages and the embedded iframe; an inline head script applies it before first paint (no flash); storage events keep parallel pages in sync. Goal-bucket colours are shared everywhere: Growth dark · Income bronze · Preservation green.

15.2 Deployment — and the one rule that matters

A GitHub repository connected to **Vercel's native Git integration**: every push to `main` redeploys the static folder (no build command, output = root). Which yields the rule that has bitten before: **committing locally is invisible — always push.** The live site only changes on push, and the sweep's attachments (the PDFs) are served from the same deploy.

16 · Replication checklist & acceptance tests

16.1 Build order

1. **Design systems** — `openbook.css` (shell, both themes) and `styles.css` (legacy).
2. `data.js` — the nine themes, the seed ideas (derivatives-first structures), the nine client books with the reconciliation invariant (§3.4), the synthetic sector history (§3.5), the MiFID keyword taxonomy (§3.6, structured checked before OTC).
3. `goals.js` — the parsers, the willingness blend, the three bucket sums, largest-remainder normalisation, provenance, `currentBuckets`, `goalAlignment` (§4, §7.2).
4. `mapping.js` — the four axes with the exact flat weights, the gate, the multiplier, tiers and gates (§5–7).
5. `scanner.js` — the 11 rules + the thin facades (§10).
6. `expressions.js` — the structure knowledge base with its keyword resolver; every structure string used anywhere must resolve.
7. **The pipeline** — `build_today_focus.py` (rubrics, validators, tactical gate, ledger), `tv_technicals.py`, `fetch_chart_series.py` (§8–9).
8. **The first sweep** — author `today_focus.json` by running the §14 loop for real; fix every warning.
9. **Support engines** — `email.js`, `charts.js`, `methodology.js` (reads PARAMS live so the in-app description can't drift), `morgan.js`.
10. **The shell** — `index.html` + `openbook.js` (§2), then the legacy pages in embed mode (`classic.html` + `app.js`, `portfolio.html`, `onepager.html`).
11. **Deploy** — GitHub → Vercel; push; verify the live URL.

16.2 Acceptance tests — run before calling it done

1. Both tabs render in both views and both themes; no console errors.
2. Gold × Amar → Gap fit **0** (Preservation bucket full); the same idea scores high for Scott / Fotis / Aurora.
3. A zero-cost-collar idea × Aurora (Retail) → fit 0, suppressed, and the MiFID reason is *shown*, not silently dropped.
4. Mitch's derived goals come out aggressive-growth from stated-risk + the book; Fotis comes out moderate from the book alone (source "revealed", confidence 0.55).
5. `build_today_focus.py` exits with zero warnings on the shipped payload; untriggered tacticals are reported as held back.
6. Re-running the build with no payload change moves **no** dates; editing one idea's thesis bumps only its `updatedAt`.
7. A chart block without a fetched series is stripped from the output.
8. Any `estimated` pillar caps the label at Medium / "High — data gap".

9. Discarding an idea in the Solutions view hides it from the Advisor view; approving restores it; a composer draft shows a DRAFT pill, maps through the live engine, and survives reload.
10. An Advisor Book tile opens the embedded portfolio report with the theme applied and a working "← All clients" return.
